TAKE-HOME ASSIGNMENT · WRITE-UP

RicherText

Three questions from the brief, answered from the code. ADR numbers refer to `docs/decisions/` in the repository.

1 How does the app decide a user is an active subscriber?

One field decides it: `customers/{uid}.plan`, which is `"paid"` or `"free"`. Only the Stripe webhook writes it, through the Admin SDK; Firestore rules deny every client write to `customers`, and a user can read only their own record. The plan is `paid` while any of the user's subscriptions is `active`, `trialing` or `past_due`. Past due keeps access on purpose: Stripe retries the card for days, and a declined card shouldn't lock someone out mid-sentence. Access ends when Stripe gives up and cancels, which arrives as a `customer.subscription.deleted` event.

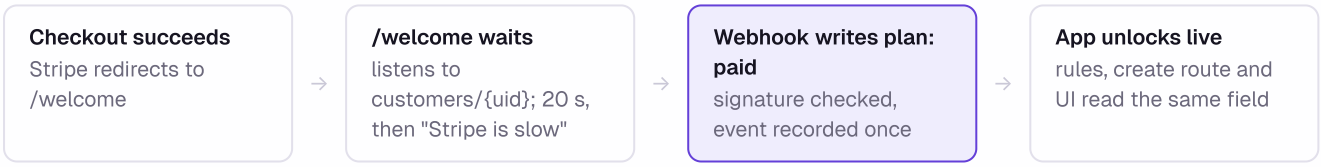
Everything that gates reads that one field, so there is no second copy to drift. Firestore rules check it for edits (a paid user edits any document; a Free user only the newest one). The create route checks it in a transaction for the Free limit. The plans page redirects a paid user away. The browser's view is a live listener on the same record, never a flag stored client-side, so a plan change shows up without a reload.

`ADR 0020` `ADR 0027` `ADR 0028` `firestore.rules` `functions/src/billing/plan.ts`

---

2 What happens if payment succeeds but the webhook is delayed?

Nothing unlocks on the redirect. The Checkout success URL is `/welcome`, a page that carries no claim about payment. It listens to `customers/{uid}` and shows "Activating your plan · Waiting for Stripe to confirm". When the webhook lands and `plan` flips to `paid`, the page names the plan and goes to Documents. After 20 seconds it switches to "Still waiting on Stripe · Your plan switches on by itself as soon as Stripe confirms" and offers a link on, so a slow webhook never traps the user on a spinner; whichever page they open next unlocks live the moment the record changes.



The webhook is built so "late" is the same as "on time". It doesn't trust the event body: it re-reads the subscription from Stripe, so events arriving out of order still end on Stripe's latest state. It records the event ID in the same transaction as the plan write, so a retried delivery is a no-op. If Stripe can't be reached, it answers 500 without recording anything, and Stripe retries. The emulator tests cover signature failures, replays, out-of-order delivery and the 500 path.

`ADR 0027` `functions/src/billing/webhook.ts` `src/app/(checkout)/welcome` `tests/emulators/stripe-webhook.test.ts`

---

3 One security decision and why

Server-held session cookies instead of client-side auth state. The browser signs in with Firebase Auth, then exchanges its short-lived ID token for a session cookie minted by the server: `httpOnly`, `Secure`, `SameSite=Lax`, 14 days. Every gated layout, page, route handler and action verifies that cookie with the Admin SDK on each request, checking signature, expiry, revocation and that the user still exists. The edge proxy only checks that a cookie is present; the real check runs where the data is read, so a forged or stale cookie is treated as signed out and cleared. Mutating routes also require a same-origin `Origin` header.

Why: the assignment's hardest requirement is that no protected action relies on a client-only check. With the cookie, nothing the browser can edit decides access: a Firestore write is accepted or refused by rules against the server's view of the user, and create, delete, Checkout and Portal go through routes that verify the cookie first. The costs are a second round-trip at sign-in and a silent refresh for long writing sessions; both are invisible to the user and worth it for a gate that can't be bypassed from DevTools.

`ADR 0026` `src/lib/auth/session.ts` `src/app/api/session` `e2e/auth.spec.ts`

Amir Masoomi · October 2026